

Introducción a la Programación con Python

Semana 5: Pruebas Unitarias y Testing

∇Nablabs

Contenido

- 1 ¿Qué es el testing?
- 2 Importancia del testing
- 3 Framework pytest
- 4 Función assert
- 5 Estructura de pruebas unitarias
- 6 Casos de prueba
- 7 Test-Driven Development (TDD)
- 8 Cobertura de código
- 9 Ejemplo integrador

¿Por qué probar el código?

- ✓ Detecta errores antes de producción
 - Garantiza que el código funciona correctamente
- ♻️ Facilita cambios y refactorización
- 🕒 Ahorra tiempo a largo plazo
 - Documenta el comportamiento esperado

 "Si no está probado, está roto"

¿Por qué hacer pruebas? ?

Sin pruebas: ❌

- Errores en producción
- Miedo a cambiar código
- Debugging manual tedioso
- No hay confianza
- Regresiones frecuentes

Con pruebas: ✔️

- Errores detectados temprano
- Refactorización segura
- Feedback automático
- Código confiable
- Regresiones prevenidas

💡 Las pruebas son una inversión

¿Qué es pytest?

Framework popular y poderoso para escribir pruebas en Python

Instalación:

```
pip install pytest
```

Ventajas de pytest:

- ✓ Sintaxis simple y clara
- ✓ Descubrimiento automático de pruebas
- ✓ Mensajes de error descriptivos
- ✓ Fixtures y parametrización
- ✓ Gran ecosistema de plugins

assert: Verificar condiciones

¿Qué hace assert?

Verifica que una condición sea verdadera; si no, lanza `AssertionError`

```
1 def sumar(a, b):  
2     return a + b  
3  
4 # Pruebas básicas con assert  
5 assert sumar(2, 3) == 5  
6 assert sumar(0, 0) == 0  
7 assert sumar(-1, 1) == 0  
8  
9 print("Todas las pruebas pasaron!")
```

➔ Salida

Todas las pruebas pasaron!

i Si algún `assert` falla, el programa se detiene

assert: Cuando falla

```
1 def multiplicar(a, b):  
2     return a + b    # Error intencional  
3  
4 # Esta prueba fallará  
5 assert multiplicar(3, 4) == 12
```

Error

AssertionError

Expected: 12

Got: 7

 El mensaje indica qué esperabas vs qué obtuviste

Archivo: calculadora.py

```
1 def sumar(a, b):  
2     """Suma dos números"""  
3     return a + b  
4  
5 def restar(a, b):  
6     """Resta dos números"""  
7     return a - b
```

Archivo: test_calculadora.py

```
1 from calculadora import sumar, restar  
2  
3 def test_sumar():  
4     assert sumar(2, 3) == 5  
5     assert sumar(0, 0) == 0  
6  
7 def test_restar():  
8     assert restar(5, 3) == 2  
9     assert restar(0, 0) == 0
```


Ejecutar pruebas con pytest ▶

En la terminal:

```
pytest test_calculadora.py
```

➔ Salida

```
===== test session starts  
=====  
collected 2 items  
  
test_calculadora.py .. [100%]  
  
===== 2 passed in 0.01s  
=====
```

✓ Cada punto (.) representa una prueba que pasó

Reglas importantes

pytest descubre automáticamente las pruebas siguiendo estas reglas:

Archivos de prueba:

- Deben empezar con `test_` o terminar con `_test.py`
- Ejemplo: `test_calculadora.py`, `utils_test.py`

Funciones de prueba:

- </> Deben empezar con `test_`
- </> Ejemplo: `test_sumar()`, `test_validacion()`

💡 Usa nombres descriptivos: `test_sumar_numeros_positivos`

Categorías importantes

- ✓ **Casos normales:** Comportamiento esperado típico
- ! **Casos límite:** Valores extremos (0, negativos, vacíos)
- 💣 **Casos excepcionales:** Errores y situaciones anormales

💡 ¡Prueba todos los escenarios!

```
1 def dividir(a, b):  
2     return a / b  
3  
4 def test_dividir_casos_normales():  
5     """Pruebas con casos típicos"""  
6     assert dividir(10, 2) == 5.0  
7     assert dividir(9, 3) == 3.0  
8     assert dividir(7, 2) == 3.5  
9     assert dividir(100, 4) == 25.0
```

i Casos que cualquier usuario típico encontraría

Casos límite !

```
1 def calcular_descuento(precio, porcentaje):
2     """Calcula descuento (0-100%)"""
3     if porcentaje < 0 or porcentaje > 100:
4         raise ValueError("Porcentaje debe estar entre 0 y 100")
5     return precio * (porcentaje / 100)
6
7 def test_descuento_casos_limite():
8     """Pruebas con valores extremos"""
9     assert calcular_descuento(100, 0) == 0
10    assert calcular_descuento(100, 100) == 100
11    assert calcular_descuento(0, 50) == 0
12    assert calcular_descuento(200, 50) == 100
```

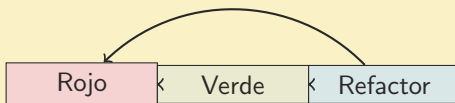
 Valores en los bordes: 0, máximo, mínimo, vacío

```
1 import pytest
2
3 def test_dividir_por_cero():
4     """Prueba que se lance excepción"""
5     with pytest.raises(ZeroDivisionError):
6         dividir(10, 0)
7
8 def test_descuento_invalido():
9     """Prueba validación de porcentaje"""
10    with pytest.raises(ValueError):
11        calcular_descuento(100, -10)
12
13    with pytest.raises(ValueError):
14        calcular_descuento(100, 150)
```

✓ `pytest.raises()` verifica que se lance la excepción correcta

¿Qué es TDD?

Metodología donde escribes las pruebas ANTES del código



-] **Rojo:** Escribe prueba que falla
-] **Verde:** Escribe código mínimo para pasar
-] **Refactor:** Mejora el código

TDD en práctica: Paso 1 - Prueba ○

Escribir la prueba primero:

```
1 # test_fibonacci.py
2 from fibonacci import fibonacci
3
4 def test_fibonacci():
5     """Prueba de números Fibonacci"""
6     assert fibonacci(0) == 0
7     assert fibonacci(1) == 1
8     assert fibonacci(2) == 1
9     assert fibonacci(5) == 5
10    assert fibonacci(10) == 55
```



Resultado

ModuleNotFoundError: No module named 'fibonacci'

○ La prueba falla porque no existe el código

Escribir código mínimo para pasar:

```
1 # fibonacci.py
2 def fibonacci(n):
3     """Retorna el n-ésimo número Fibonacci"""
4     if n <= 0:
5         return 0
6     elif n == 1:
7         return 1
8     else:
9         a, b = 0, 1
10        for _ in range(2, n + 1):
11            a, b = b, a + b
12        return b
```

✓ Resultado

```
===== 1 passed in 0.01s
=====
```

Mejorar el código (opcional):

```
1 def fibonacci(n):
2     """Retorna el n-ésimo número Fibonacci (optimizado)"""
3     "
4     if n <= 0:
5         return 0
6     elif n == 1:
7         return 1
8
9     # Uso de tuplas para asignación eficiente
10    a, b = 0, 1
11    for _ in range(2, n + 1):
12        a, b = b, a + b
13    return b
```

✔ Las pruebas siguen pasando, código más limpio

¿Qué es la cobertura?

Mide qué porcentaje del código es ejecutado por las pruebas

Instalar pytest-cov:

```
pip install pytest-cov
```

Ejecutar con cobertura:

```
pytest --cov=calculadora test_calculadora.py
```

i La cobertura NO garantiza calidad, pero ayuda a identificar código no probado

Reporte de cobertura

Ejemplo de reporte

```
----- coverage: platform win32, python 3.11 -----  
Name Stmts Miss Cover  
-----  
calculadora.py 10 1 90%  
test_calculadora.py 15 0 100%  
-----  
TOTAL 25 1 96%
```

Reporte HTML detallado:

```
pytest --cov=calculadora --cov-report=html
```

💡 Genera carpeta htmlcov/ con reporte visual

Recomendaciones

- ✓ Una prueba = un comportamiento específico
- ✓ Nombres descriptivos de pruebas
- ✓ Pruebas independientes (no dependen entre sí)
- ✓ Pruebas rápidas y determinísticas
- ✓ Probar casos normales, límite y excepciones
- ✓ Mantener pruebas actualizadas
- ✓ Ejecutar pruebas frecuentemente
- ✓ 100 % de cobertura es ideal, 80 %+ es bueno

Archivo: validador.py

```
1 def validar_password(password):
2     """
3     Valida que la contraseña cumpla requisitos:
4     - Al menos 8 caracteres
5     - Contiene mayúsculas y minúsculas
6     - Contiene al menos un número
7     """
8     if len(password) < 8:
9         return False
10
11     tiene_mayuscula = any(c.isupper() for c in password)
12     tiene_minuscula = any(c.islower() for c in password)
13     tiene_numero = any(c.isdigit() for c in password)
14
15     return tiene_mayuscula and tiene_minuscula and
    tiene_numero
```

Archivo: test_validador.py

```
1 from validador import validar_password
2
3 def test_password_valido():
4     """Casos normales válidos"""
5     assert validar_password("Password123") == True
6     assert validar_password("MiClave99") == True
7
8 def test_password_muy_corto():
9     """Caso límite: menos de 8 caracteres"""
10    assert validar_password("Abc123") == False
11
12 def test_password_sin_mayuscula():
13     """Falta mayúscula"""
14     assert validar_password("password123") == False
15
16 def test_password_sin_numero():
17     """Falta número"""
18     assert validar_password("Password") == False
```

Proyecto: Ejecutar pruebas

```
pytest test_validador.py -v
```

➔ Salida

```
===== test session starts
=====
test_validador.py::test_password_valido PASSED [ 25%]
test_validador.py::test_password_muy_corto PASSED [ 50%]
test_validador.py::test_password_sin_mayuscula PASSED [ 75%]
test_validador.py::test_password_sin_numero PASSED [100%]

===== 4 passed in 0.02s
=====
```

- ✔ La opción `-v` muestra detalles de cada prueba

Comandos útiles de pytest >_

```
# Ejecutar todas las pruebas
pytest

# Ejecutar con verbosidad
pytest -v

# Ejecutar archivo específico
pytest test_calculadora.py

# Ejecutar prueba específica
pytest test_calculadora.py::test_sumar

# Detener en primer error
pytest -x

# Mostrar prints
pytest -s

# Con cobertura
pytest --cov=modulo
```

¿Qué aprendimos hoy?

- ✓ Importancia del testing
- ✓ Framework pytest
- ✓ Función assert para verificaciones
- ✓ Estructura de pruebas unitarias
- ✓ Casos de prueba: normales, límite, excepciones
- ✓ Test-Driven Development (TDD)
- ✓ Cobertura de código
- ✓ Buenas prácticas de testing

 ¡Prueba tu código siempre!



¡Preguntas!

Gracias por su atención